(Windy) Real-Time Weather Monitoring Display using ESP32C6

Aim:

To design and implement an IoT-based real-time weather monitoring system using the ESP32C6 that fetches weather data from the Open Weather Map API via Wi-Fi and displays it on an I2C LCD module.

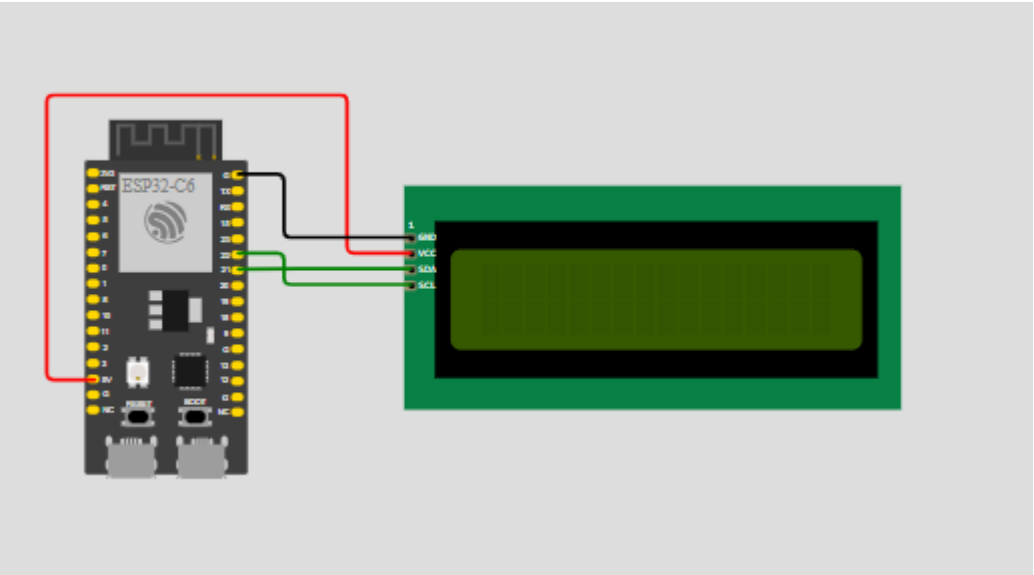
Components Required:

Sno	Components	Quantity
1	ESP32C6 Dev Module	1
2	USB to type – c cable	1
3	LCD Display	1
4	Jumper Wires	As needed

Working Principle:

The ESP32C6 connects to a Wi-Fi network and sends an HTTP request to the Open Weather Map API. The API returns weather data in JSON format, which is processed by the ESP32C6. The extracted information such as temperature, humidity, and weather condition is displayed on the LCD screen.

Circuit Diagram:



Code:

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <Arduino_JSON.h>
#include <LiquidCrystal_I2C.h>
#include <Wire.h>

/* ================= WiFi Credentials ================= */
const char* ssid     = GIVE_YOUR_SSID";
const char* password = "GIVE_YOUR_PASS";

/* ================= OpenWeatherMap ================= */
String apiKey = "f321054b786715e297d668f9bcbfe1f2";
String city = "Tiruchirappalli";
String countryCode = "IN";

/* ================= Timing ================= */
unsigned long lastTime = 0;
unsigned long timerDelay = 10000;   // 10 seconds

/* ================= LCD ================= */
#define SDA_PIN 21
#define SCL_PIN 22

LiquidCrystal_I2C lcd(0x27, 16, 2);

/* ================= NTP ================= */
const char* ntpServer = "pool.ntp.org";
long gmtOffset_sec = 0;
int daylightOffset_sec = 0;

/* ================= Variables ================= */
String jsonBuffer;

/* ============================================== */

void setup() {
  Serial.begin(115200);

  /* I2C Init for ESP32-C6 */
  Wire.begin(SDA_PIN, SCL_PIN);

  lcd.init();
  lcd.backlight();
```

```
  /* WiFi Connection */
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi connected");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}

void loop() {
  if (millis() - lastTime > timerDelay) {

    if (WiFi.status() == WL_CONNECTED) {

      String serverPath =
        "http://api.openweathermap.org/data/2.5/weather?q=" +
        city + "," + countryCode +
        "&units=metric&APPID=" + apiKey;

      jsonBuffer = httpGETRequest(serverPath.c_str());
      Serial.println(jsonBuffer);

      JSONVar weatherData = JSON.parse(jsonBuffer);

      if (JSON.typeof(weatherData) == "undefined") {
        Serial.println("JSON Parsing Failed");
        return;
      }

      /* Time zone from API */
      gmtOffset_sec = weatherData["timezone"];
      configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);

      String currentTime = getLocalTimeString();

      /* LCD Display */
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print(currentTime);
      lcd.print(" ");
      lcd.print(weatherData["weather"][0]["main"]);
```

```
    lcd.setCursor(0, 1);
    lcd.print("T:");
    lcd.print(weatherData["main"]["temp"]);
    lcd.print((char)223); // Degree symbol
    lcd.print("C ");

    lcd.print("H:");
    lcd.print(weatherData["main"]["humidity"]);
    lcd.print("%");
   }
   else {
    Serial.println("WiFi Disconnected");
   }

   lastTime = millis();
  }
}

/* ================= Time Function ================= */
String getLocalTimeString() {
  struct tm timeinfo;

  if (!getLocalTime(&timeinfo)) {
   return "00:00";
  }

  char timeStr[6];
  strftime(timeStr, sizeof(timeStr), "%H:%M", &timeinfo);
  return String(timeStr);
}

/* ================= HTTP GET ================= */
String httpGETRequest(const char* serverName) {
  WiFiClient client;
  HTTPClient http;

  http.begin(client, serverName);
  int httpResponseCode = http.GET();

  String payload = "{}";

  if (httpResponseCode > 0) {
   payload = http.getString();
  } else {
   Serial.print("HTTP Error: ");
   Serial.println(httpResponseCode);
  }
```

```
  http.end();
  return payload;
}
```

Tasks:

• Connect ESP32C6 to Wi-Fi and verify internet connectivity by printing the IP address in the Serial Monitor

• Fetch real-time weather data from OpenWeatherMap API using HTTP GET request

• Parse the JSON response to extract temperature, humidity, and weather condition

• Display the extracted weather data on an I2C LCD in a readable format

• Implement periodic updates (e.g., every 10 seconds) and handle errors like no internet or invalid API response